

COMPUTACIÓN MÓVIL · CIENCIAS DE LA COMPUTACIÓN



Sistema para Restaurante

Dashboard web + Aplicación móvil

INTEGRANTES

Farrés, Martín
Suden, Paulina
Yudica, Franco
Zangla, Adrián

FING — UNCuyo
Mendoza, Argentina
github.com/Stack-Underflow-UNCuyo

Contenido de la presentación

01

Problema

Qué resuelve el sistema: app de mozo, app de cocinero y dashboard.

02

Tecnología Backend

Arquitectura MVC en Spring Boot y el patrón de Mappers genéricos.

03

Seguridad

JWT + Spring Security + BCrypt: flujo de autenticación y autorización.

04

Tecnología usadas en aplicación móvil

React Native + Expo: arquitectura, componentes y hooks.

05

Historias de usuario

06

Diagrama de clases

07

Diagrama de entidad-relación

08

Cache offline

useQuery + AsyncStorage: stale-while-revalidate.

09

Mercado Pago

Cobro presencial por QR dinámico: flujo end-to-end.

10

Demo en vivo

El problema a resolver

¿Cómo digitalizar la operación diaria de un restaurante?



App del Mozo

- Ver el estado de las mesas
- Tomar la comanda en el momento
- Aplicar promociones
- Cobrar (efectivo o QR de MP)
- Registrar reseñas del cliente



App del Cocinero

- Kanban de comandas en tiempo real
- Avanzar estados

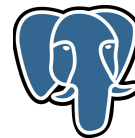
Carta!



Dashboard Admin

- Gestión de carta, secciones y artículos
- Empleados, mesas
- Reportes y métricas

Arquitectura



Maven™

CAPAS



business/domain/

Entidades + DTO + Enumeraciones



business/repository/

Acceso a datos (Spring Data JPA). Una interfaz por entidad.



business/logic/service/

Reglas de negocio. Transacciones y validaciones.



business/mapper/

Conversiones Entity ↔ DTO. BaseMapper define el contrato común.



controllers/

Endpoints REST (@RestController). Reciben el pedido y devuelven DTOs.

JWT + Spring Security + BCrypt



JWT

Token firmado



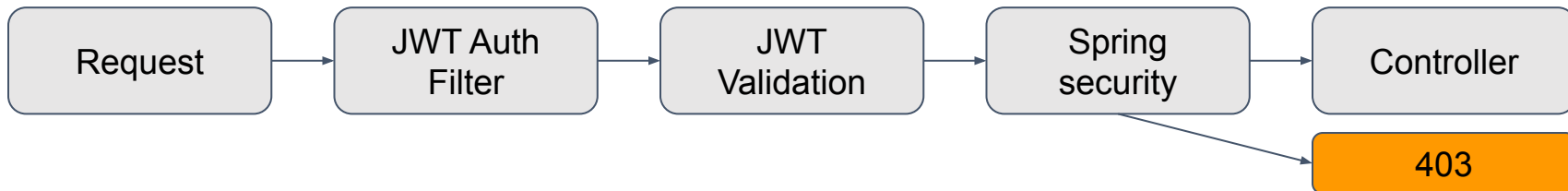
Spring Security

Filtros + roles



BCrypt

Hash de contraseña



Arquitectura React Native + Expo

Stack: React Native 0.85 · Expo SDK 56 · Expo Router (file-based) · TypeScript

ESTRUCTURA src/

app/	<i>rutas (Expo Router)</i>
(mozo) /	<i>grupo mozo: mesas, comandas, etc.</i>
(cocinero) /	<i>grupo cocinero: kanban</i>
login.tsx	<i>pantalla pública</i>
_layout.tsx	<i>AuthGate + providers</i>
controllers/	
context/	<i>AuthContext, CartContext</i>
hooks/	<i>lógica separada y reutilizable</i>
models/	
types/	<i>tipos TypeScript del dominio</i>
services/	<i>llamadas HTTP al backend</i>
views/	
constants/	<i>UI reutilizable por dominio</i>
components/	<i>UI reutilizable por dominio</i>
lib/	<i>config. de herramientas de terceros</i>

CONCEPTOS CLAVE



Expo Router (file-based)

Cada archivo en **app/** es una **ruta**. Los grupos **(mozo)** y **(cocinero)** aíslan navegación por **rol sin afectar la URL**.



Componentes funcionales + hooks

useState/useEffect para UI local; **hooks** de dominio (useCartas, useMesas) para datos. Pantalla = composición de componentes.

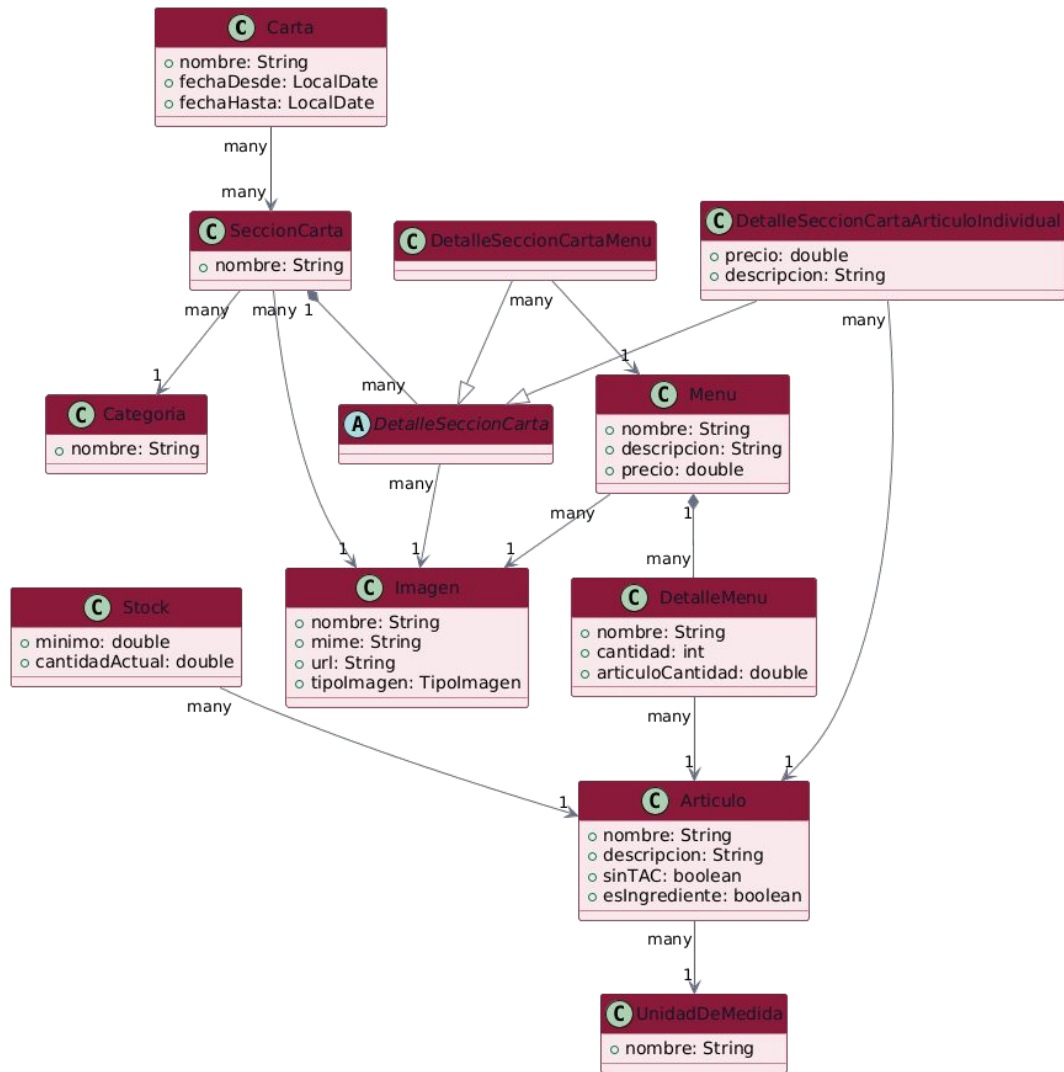


AuthContext global

Token persistido en **expo-secure-store** (cifrado). **AuthGate** redirige por rol: MOZO → (mozo), COCINERO → (cocinero).

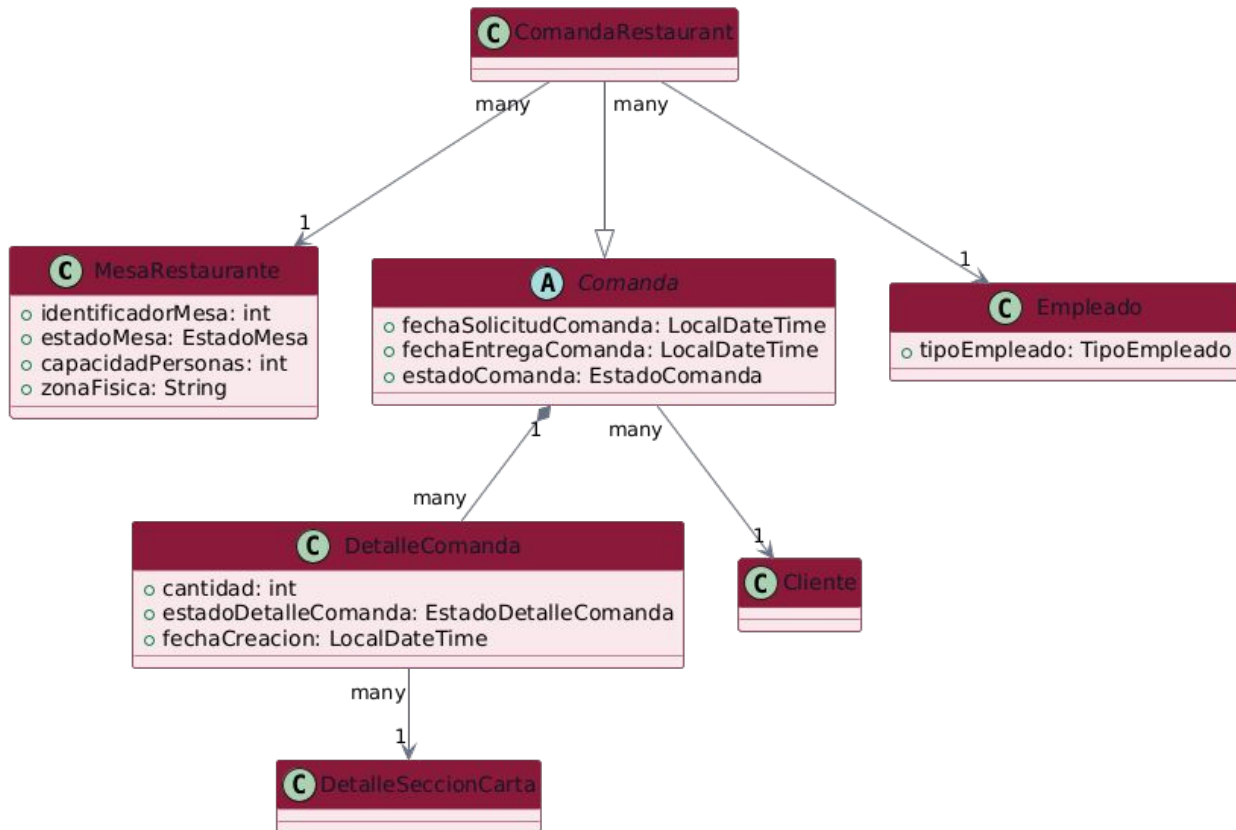
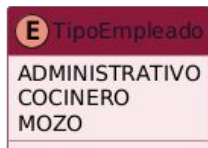
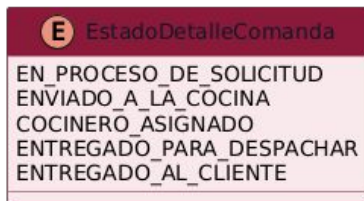
Mozo: Menu

Historia de usuario	Visualizar carta digital en la aplicación	ID	2
Prioridad	1	Estimación	5h
Descripción	Como: mozo del restaurante		
	Quiero: visualizar la carta completa y actualizada de productos y categorías desde la aplicación móvil		
	Para: poder informar a los clientes sobre las opciones disponibles, detalles y precios al momento de tomar su pedido		
Criterio de Aceptación	• Cuando accedo a la sección de "Carta", entonces la app muestra todas las categorías de productos (entradas, platos principales, postres, bebidas). • Cuando selecciono un producto específico, entonces puedo ver sus detalles (nombre, descripción, precio y foto si aplica). • Cuando un producto se queda sin stock en el sistema, entonces se muestra deshabilitado o con un indicador de "Sin stock" en la carta del mozo.		
Comentarios		Dependencias	



Mozo: Gestión Comandas

Historia de usuario	Gestión de comandas por mesa	ID	3
Prioridad	1	Estimación	8h
Descripción	Como: mozo del restaurante		
	Quiero: poder visualizar, generar y editar las comandas asociadas a las mesas que atiendo		
	Para: registrar de forma exacta el pedido de los clientes y que esta información llegue a la cocina correctamente		
Criterio de Aceptación	• Cuando selecciono una mesa ocupada que no tiene pedidos, entonces la app me permite crear una nueva comanda agregando ítems de la carta. • Cuando selecciono una mesa con una comanda activa, entonces veo el detalle completo de los ítems ya pedidos y sus cantidades. • Cuando necesito modificar un pedido, entonces puedo agregar nuevos ítems a la comanda existente (los ítems ya enviados a cocina no se pueden eliminar sin autorización).		
Comentarios		Dependencias	HDU 2 (Ver Carta)



Mozo: Gestión de Mesas

Historia de usuario	Control y actualización del estado de las mesas	ID	4
Prioridad	2	Estimación	6h
Descripción	Como: mozo del restaurante		
	Quiero: visualizar el estado de todas las mesas y poder actualizarlo (Ej: Libre, Ocupada, Esperando comida)		
	Para: llevar un control visual rápido del salón y saber qué mesas requieren atención inmediata		
Criterio de Aceptación	• Cuando ingreso a la vista principal de mesas, entonces veo la distribución o listado con colores/iconos según su estado actual. • Cuando llegan clientes a una mesa libre, entonces puedo seleccionarla y cambiar su estado manualmente a "Ocupada". • Cuando una mesa finaliza su servicio y se limpia, entonces puedo cambiar su estado nuevamente a "Libre".		
Comentarios		Dependencias	

C MesaRestaurante

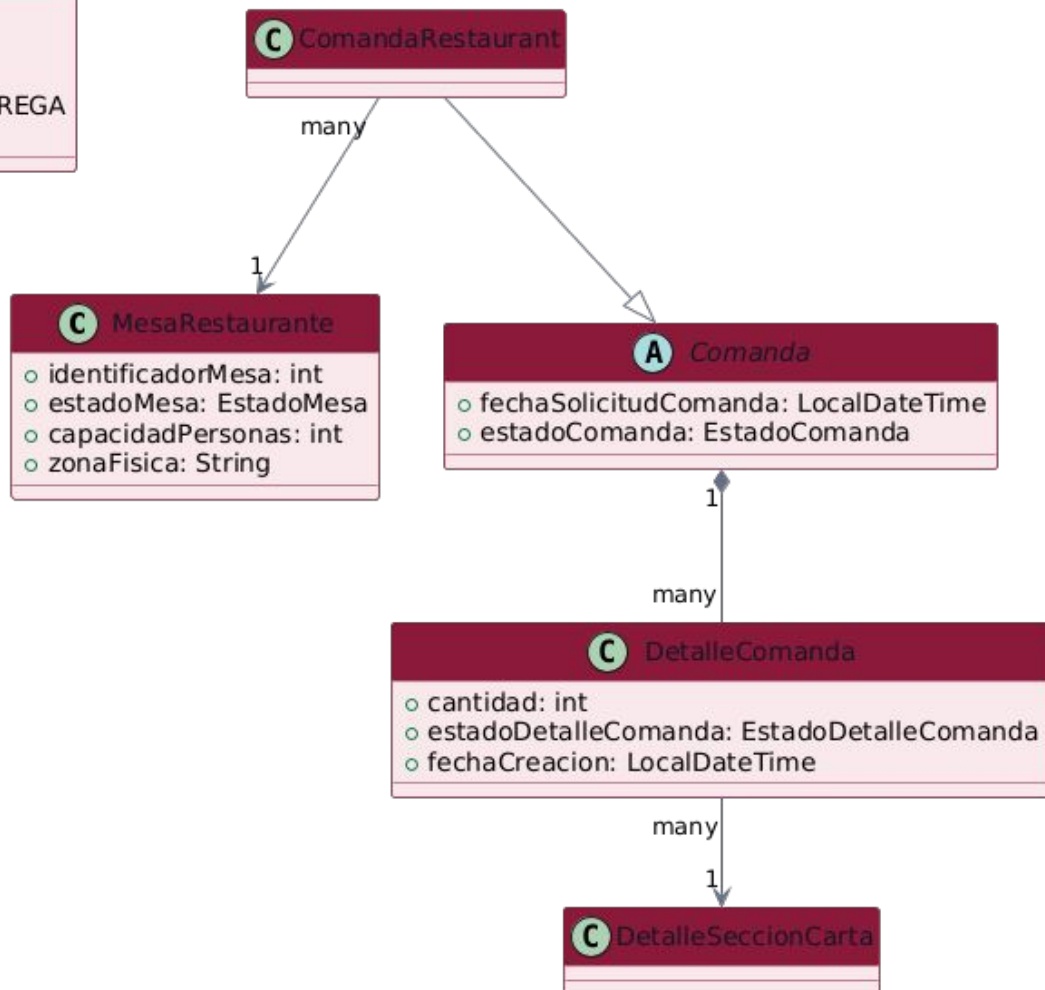
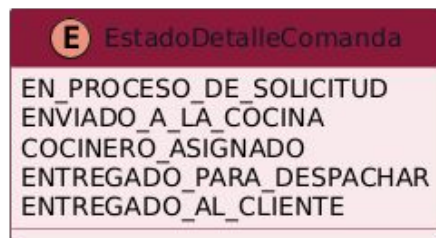
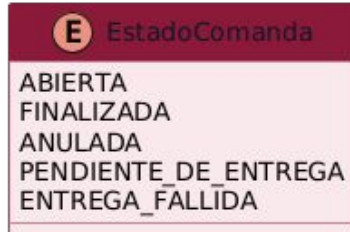
- o identificadorMesa: int
- o estadoMesa: EstadoMesa
- o capacidadPersonas: int
- o zonaFisica: String

E EstadoMesa

LIBRE
RESERVADA
OCUPADA
FUERA_DE_SERVICIO

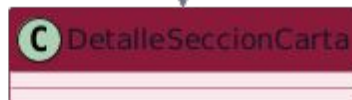
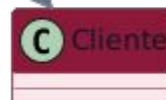
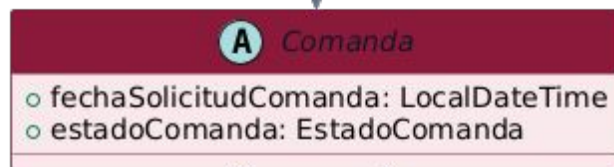
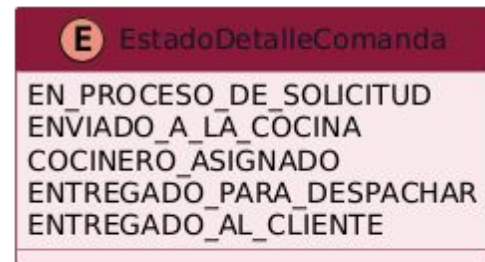
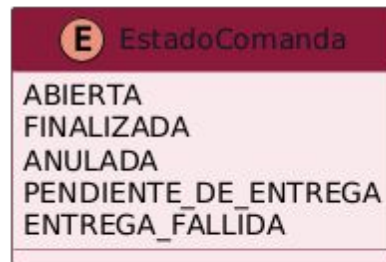
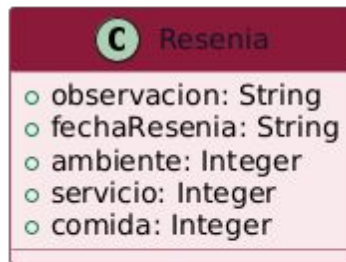
Mozo: Gestión de Platos

Historia de usuario	Seguimiento y entrega de platos	ID	5
Prioridad	2	Estimación	7h
Descripción	Como: mozo del restaurante		
	Quiero: ver el estado de preparación de los ítems de una comanda y poder marcarlos como entregados		
	Para: saber cuándo retirar la comida de la cocina y confirmar que el cliente ya recibió su pedido		
Criterio de Aceptación	• Cuando la cocina marca un ítem como "Listo", entonces la aplicación del mozo muestra una alerta o indicador visual en la mesa correspondiente. • Cuando visualizo el detalle de la comanda, entonces veo claramente qué platos están "En preparación", "Listos para entregar" y "Entregados". • Cuando sirvo el plato en la mesa, entonces puedo presionar un botón para cambiar el estado de ese ítem a "Entregado".		
Comentarios		Dependencias	HDU 3 (Generar comanda)



Mozo: Registro de Reseña

Historia de usuario	Registro de reseña y calificación del servicio	ID	6
Prioridad	3	Estimación	4h
Descripción	Como: mozo del restaurante		
	Quiero: poder registrar una calificación y reseña oral brindada por el cliente al finalizar la comida		
	Para: que el restaurante pueda recopilar feedback valioso sobre la atención y la calidad de los platos		
Criterio de Aceptación	• Cuando la mesa solicita la cuenta o está por finalizar el servicio, entonces la app me habilita un formulario rápido de reseña. • Cuando el cliente me da su feedback, entonces puedo ingresar una puntuación (ej. 1 a 5 estrellas) y opcionalmente escribir un comentario de texto. • Cuando guardo la reseña, entonces esta queda asociada a la comanda histórica para futuras estadísticas del local.		
Comentarios	Puede realizarse al momento del cobro	Dependencias	HDU 3 y HDU 4



many

1

many

1

many

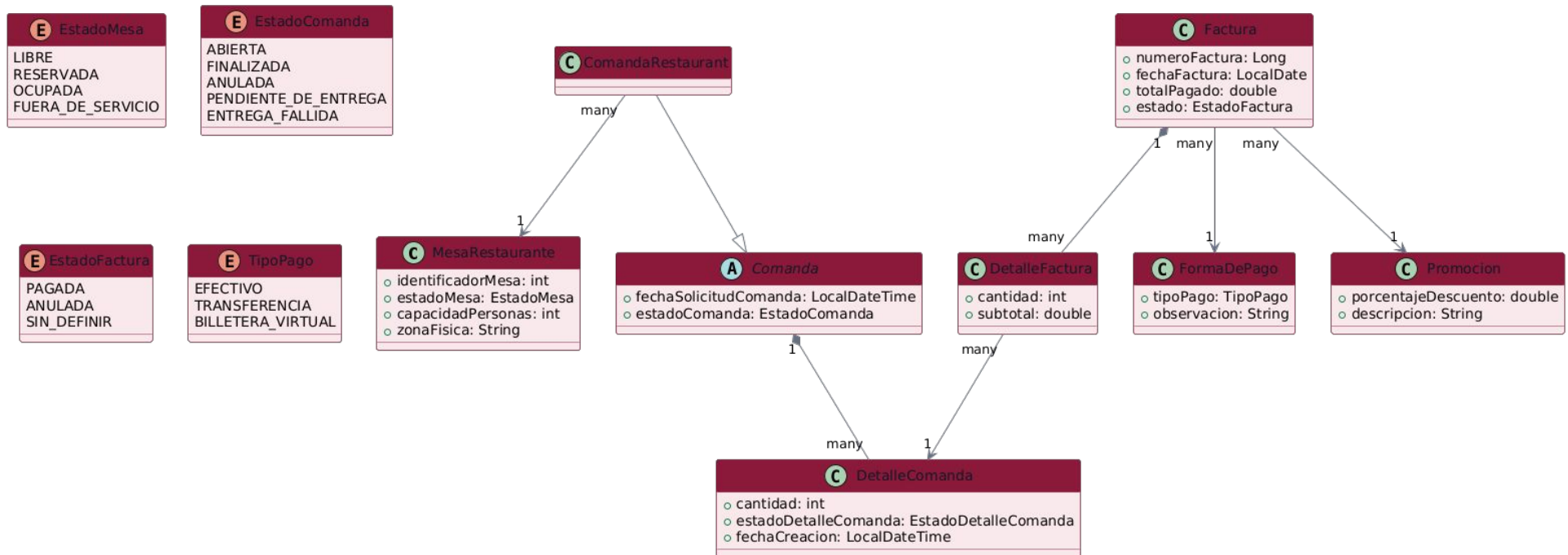
1

many

1

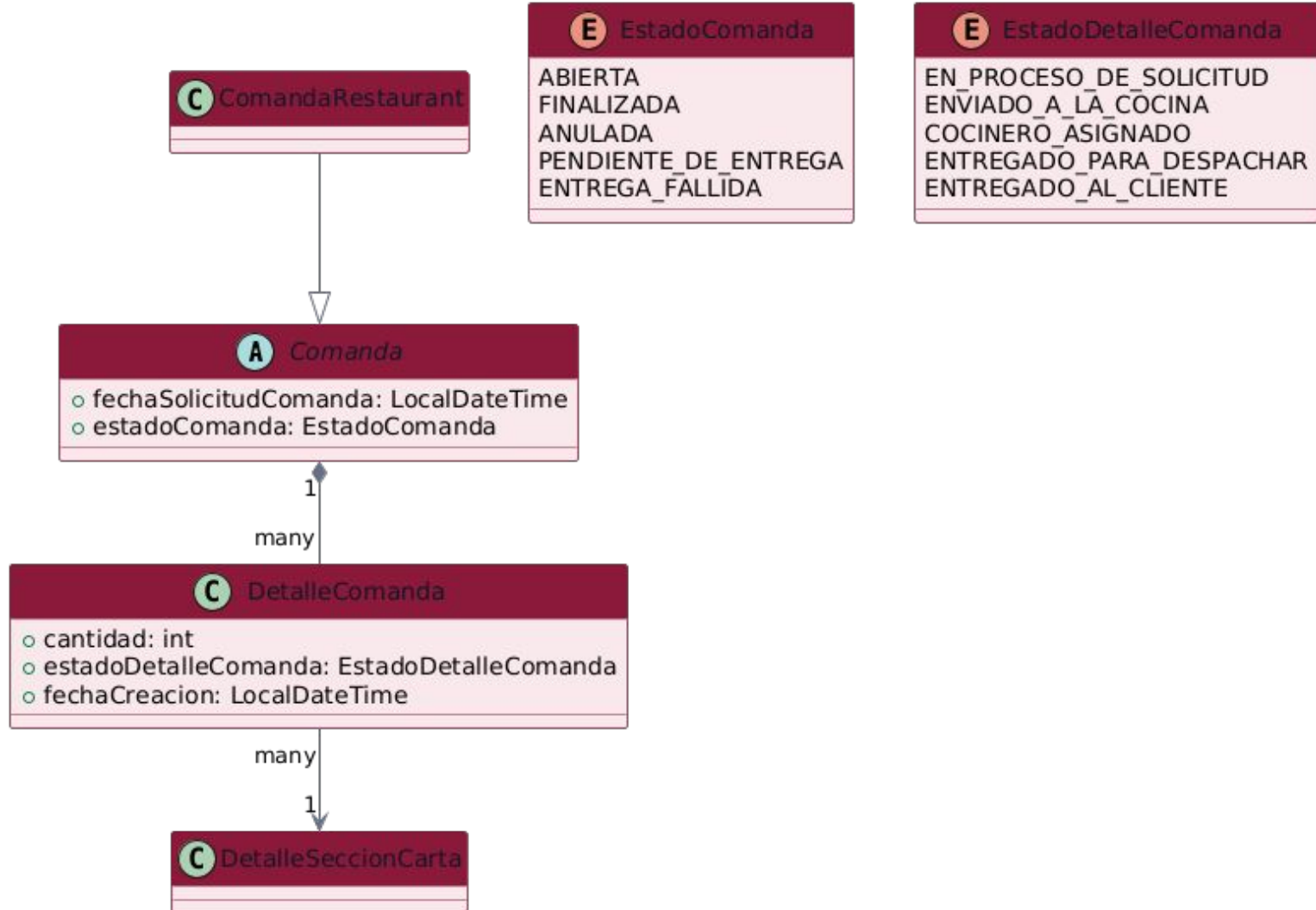
Mozo: Mercado Pago y generación de factura

Historia de usuario	Cobro presencial mediante QR de Mercado Pago	ID	7
Prioridad	4	Estimación	9h
Descripción	Como: mozo del restaurante		
	Quiero: generar un código QR de Mercado Pago con el monto de la mesa ya cargado		
	Para: que el cliente pague desde su celular escaneando dicho QR y generar la factura		
Criterio de Aceptación	• Cuando selecciono "Mercado Pago" como medio de pago y toco "Generar QR de cobro", entonces la app muestra un QR dinámico con el monto exacto del consumo. La app detecta el pago dentro de los 4 segundos siguientes sin que el mozo tenga que hacer nada. • Cuando el backend recibe la confirmación, entonces se genera una factura en estado PAGADA con forma de pago TRANSFERENCIA, la comanda pasa a FINALIZADA y la mesa queda LIBRE. • Cuando se intenta generar un QR sin la configuración inicial cargada, entonces el backend devuelve un error claro.		
Comentarios		Dependencias	



Cocinero: Gestión de las comandas en cocina

Historia de usuario	Gestionar el estado de preparación de las comandas en cocina	ID	8
Prioridad	1	Estimación	8h
Descripción	Como: cocinero del restaurante		
	Quiero: visualizar los pedidos entrantes y actualizar el estado de preparación de cada ítem (ej. Pendiente, En preparación, Listo)		
	Para: organizar el flujo de trabajo en la cocina y notificar automáticamente al mozo cuando un plato debe ser servido		
Criterio de Aceptación	• Cuando el mozo genera o envía una comanda, entonces los ítems aparecen automáticamente en la pantalla de la cocina. • Cuando comienzo a preparar un ítem, entonces puedo cambiar su estado a "En preparación" (ej. moviéndolo de columna en un tablero Kanban). • Cuando termino de cocinar un ítem, entonces puedo marcarlo como "Listo", lo que emite una alerta o actualización en la vista del mozo.		
Comentarios	Se sugiere implementar mediante una vista de tipo tablero Kanban para mayor agilidad visual.	Dependencias	HDU 3 (Ver / Generar comanda)



Administrador: Gestión de Usuario

Historia de usuario	Crear, editar y listar usuarios en el sistema	ID	8
Prioridad	1	Estimación	8h
Descripción	Como: administrador del restaurante		
	Quiero: poder crear, editar y visualizar el listado de todos los usuarios del personal		
	Para: gestionar de manera centralizada los accesos, roles y credenciales de los empleados en la plataforma		
Criterio de Aceptación	• Cuando accedo a la sección de "Usuarios", entonces el panel muestra un listado completo con el nombre, correo electrónico, estado y rol (Mozo, Cocinero, Administrador) de cada cuenta. • Cuando completo el formulario de registro con datos válidos (nombre, email, contraseña y rol), entonces el sistema crea el usuario y este ya queda habilitado para iniciar sesión. • Cuando selecciono un usuario del listado y modifico sus datos o su rol, entonces los cambios se guardan correctamente e impactan de inmediato en sus permisos de acceso.		
Comentarios		Dependencias	

Administrador: Gestión de mesas

Historia de usuario	Crear, editar y listar mesas del establecimiento	ID	9
Prioridad	1	Estimación	6h
Descripción	Como: administrador del restaurante		
	Quiero: poder dar de alta, modificar y listar las mesas del salón en el sistema		
	Para: mantener actualizada la distribución del salón y permitir que los mozos puedan asignar comandas correctamente		
Criterio de Aceptación	• Cuando ingreso al módulo de "Mesas", entonces visualizo un listado estructurado de todas las mesas configuradas con su número identificador y capacidad de comensales. • Cuando añado una nueva mesa ingresando su número único y capacidad, entonces se guarda en la base de datos y se vuelve visible en la interfaz operativa del salón. • Cuando edito las propiedades de una mesa (ej. modificar su número o deshabilitarla temporalmente por mantenimiento), entonces los cambios se actualizan en tiempo real para evitar que se le asignen pedidos erróneamente.		
Comentarios		Dependencias	

Cache offline · useQuery + AsyncStorage

El problema: antes la app era 100% online — sin red, pantalla vacía o error. *Ahora los datos de solo lectura se cachean en disco.*



1. Cache en memoria

React Query guarda el resultado de cada query. Otra pantalla pide lo mismo → respuesta instantánea.



2. Persistencia

asyncStoragePersister escribe el cache en AsyncStorage. Al reabrir la app, se rehidrata desde disco.



3. Stale-while-revalidate

Muestra el dato cacheado al instante (sin spinner) y, si hay red, lo refresca de fondo.

CONFIG + HOOK

```
// queryClient.ts
staleTime: 5 * 60_000,
gcTime: 24 * 3600_000,
retry: 1

// useCartas.ts
useQuery({
  queryKey: ["cartas"],
  queryFn: getCartas
});
```

⚠ Limitación a propósito: las escrituras (cobrar, dejar reseña) no se cachean ni se encolan offline — *implementarlo implicaría riesgo de cobros duplicados.*

Mercado Pago

Primer paso: Credenciales de prueba



SETUP: crear **Sucursal** + **Caja** en MP



1 **Sucursal**

Establecimientos físicos registrados en Mercado Pago y pueden tener una o más cajas vinculadas

2 **Caja**

Puntos de venta vinculado a una sucursal garantizando la conciliación de pagos por Código QR en establecimientos físicos

Mercado Pago - Flujo de cobro



CREAR ORDERS: pago puntual por un monto fijo, vinculado a una caja, que el cliente paga escaneando el **QR** que **MP** genera para esa orden.

Modelo: QR dinámico (uno distinto por cobro) · **Confirmación:** polling cada 4s

1	App mozo	POST /mercadopago/orders	Crea la order con monto. MP devuelve qrData (EMVCo).
2	App mozo	Renderiza el QR	Con react-native-qrcode-svg.
3	Cliente	Escanea y paga	App de Mercado Pago del cliente
4	App mozo	Polling cada 4s	POST /orders/{id}/confirmar-pago hasta que se confirma el pago
5	Backend	Genera factura	factura PAGADA + comanda FINALIZADA + mesa LIBRE (idempotente).

Mercado Pago



IDEMPOTENCIA: Repetir una acción que ya se realizó devuelve el mismo valor que la primera vez

X-Idempotency-Key (UUID) en el **POST /orders** a MP: header **obligatorio** que evita doble cargo si se reintenta el request

```
headers.add("X-Idempotency-Key", UUID.randomUUID().toString());
```

Lo mismo sucede con la generación de la factura: Verifica si la comanda ya fue facturada antes de generar. :

```
public boolean comandaYaFacturada(String comandaId) {  
  
    return  
    detalleFacturaService.comandaYaFacturada(comandaId);    }
```

04 · DASHBOARD WEB + APLICACIÓN MÓVIL

Bugs en vivo

Next.js · TypeScript · *React Native*



¿PREGUNTAS?

Gracias

Stack Underflow · FING UNCuyo

`github.com/Stack-Underflow-UNCuyo/restaurant-mobile-app`